

Calcul Parallèle

TP : Partie 1

Thomas STAVIS
(thomas.stavis@ens-lyon.fr)

Jonathan Rouzaud-Cornabas
(jonathan.rouzaud-cornabas@inria.fr)

This TP deals theoretically with parallelism applied to certain problems. It is means to understand the usefulness of parallelism, to know how to implement it, as well as to introduce some of its subtleties and limits. This TP has 2 parts : a on the sorts, and another one on the multiplication of matrices.

Submitting A report will need to be submitted for this lab. It is advisable to start writing it during the sessions then to take it back to your place before returning it. As the lab is in 2 parts, you will need to submit 2 reports. You will need to send these reports in the same email at the address "thomas.stavis@ens-lyon.fr", with the object "[HPC GPU INSA] : <first name> <name>". The deadline for receipt of this report is 10/12/2026 at 6 PM. Even if you can do this TP in a group during the sessions and outside, the reports are individual.

Notation Part 1 of this TP is based on 28 points. The score for this part will be *minimum*(20, your score to the *part*)/20. The final grade of TP1 will be an average of the grades of the 2 parts.

LLM The use of LLM to answer questions is not penalized. However, the TP is not long and its goal is for you to integrate certain concepts. If you help yourself with it, please note it in the reports how you used it.

0 Recal : Complexity in time

The **complexity in time** is a theoretical measure of the time required to execute an algorithm. Depending on the size of the parameters given as input (list size, matrix, etc.), it counts the number of assignments, comparisons and arithmetic operations performed during this algorithm. Then, assuming that each of these operations takes a constant time C ; *seconds* to be executed, we can deduce the execution time of the algorithm $Time = Number\ operations \times C\ sec$. We note $f(n_1, \dots, n_m)$ the function that, for $m \in \mathbb{N}$ parameter sizes, returns the complexity in time.

We will particularly focus on the **complexity** in the worst case. The principle is to count the maximum number of operations in every situation : a loop runs from start to finish and it is always the conditional branch that requires the most operations that runs.

Exemple

We will calculate the complexity in time in the worst case for this simple algorithm, which runs through a list of numbers and, if the number and pair, adds 1 to that number.

Algorithm 0: Ajout de 1

Data: L liste, n taille de L

```
1 for  $i = 0$  to  $n - 1$  do
2    $c \leftarrow L[i]$ 
3   if  $c \% 2 == 0$  then
4      $L[i] \leftarrow c + 1$ 
5 return  $L$ 
```

Let's analyze the algorithm 0 :

- line 1 : the initialization counts for 1 operation, then all the lines in the body of the loop will be executed n times :
 - line 2 : 1 operation (affectation)
 - line 3 : 2 operations (modulo and comparison)
 - line 4 : 2 operations (addition and allocation), we count it for each loop turn to respect the worst case
 - line 1 : the loop maintenance counts for 4 operations ($i = i + 1$ and $i \leq n - 1$)
- line 5 : the return counts for 1 operation.

The complexity of the Algorithm 0 is then $f(n) = 1 + (n - 1) \times (1 + 2 + 2 + 4) + 1$. That gives $f(n) = 4n - 7$.

Notation "big $\mathcal{O}$ "

In this TP, we are going to work with large sizes of parameters. When this is the case, the additive and multiplicative constants of the complexities have a lesser impact compared to the parameter sizes : it is rather the latter that will define the **asymptotic behavior** of the complexity. We will then use the notation $\mathcal{O}(\Gamma(f(n)))$, with Γ returning the dominant member of $f(n)$ (see asymptotic comparison scales¹), when we compare the complexities of different algorithms.

To resume the complexity analysis with the "big $\mathcal{O}$ " for algorithm 0, we have :

- ligne 1 : $\mathcal{O}(1)$, puis $\mathcal{O}(n)$ fois :
 - ligne 2 : $\mathcal{O}(1)$
 - ligne 3 : $\mathcal{O}(1)$
 - ligne 4 : $\mathcal{O}(1)$
 - ligne 1 : $\mathcal{O}(1)$ $\left. \vphantom{\begin{array}{l} \text{— ligne 2 : } \mathcal{O}(1) \\ \text{— ligne 3 : } \mathcal{O}(1) \\ \text{— ligne 4 : } \mathcal{O}(1) \\ \text{— ligne 1 : } \mathcal{O}(1) \end{array}} \right\} \text{équivalent à } \mathcal{O}(1)$
- ligne 5 : $\mathcal{O}(1)$

That gives a complexity of $\mathcal{O}(1) + \mathcal{O}(n) \times \mathcal{O}(1) + \mathcal{O}(1)$, and then in $\mathcal{O}(n)$.

1. https://fr.wikipedia.org/wiki/Comparaison_asymptotique

1 Sorting algorithms (/28)

In this part, we will theoretically apprehend the usefulness of parallelism via two sorting algorithms : selection sorting and merge sorting. We will consider only lists of integers, without duplicates.

1.1 Selection sorting (/16)

The principle of selection sorting is to go through a list to find its minimum, place it at the top of the list, and then repeat the process on the unsorted part of the list. Here is the pseudo-code of this algorithm :

Algorithm 1: Selection sorting (sequential)

Data: L liste, n taille de L

```
1 for  $i = 0$  to  $n - 2$  do
     $min \leftarrow i$ 
    for  $j = i + 1$  to  $n - 1$  do
        if  $L[j] < L[min]$  then
5          $min \leftarrow j$ 
     $tmp \leftarrow L[min]$ 
     $L[min] \leftarrow L[i]$ 
     $L[i] \leftarrow tmp$ 
9 return  $L$ 
```

Question 1 (/1,5)

For each line, write the number of operations (assignments, comparisons and arithmetic) in the worst case.

Question 2 (/1,5)

Compute the number of operations Algorithm 1 do in the worst case. Deduce the complexity in time, with the "big $\mathcal{O}$ " notation.

Question 3 (/4)

We now run this algorithm using 3 threads, 1 main thread A and 2 secondary threads B and C. The algorithm behaves as follows :

- At the same time, B and C search for respectively the 1st and 2nd smallest items in the list, save them in global variables, send a signal to A, then wait for a signal from A before restarting the search on the new list.
- When A receives the 2 signals, it proceeds sequentially to exchange the 2 smallest numbers at the beginning of the list, then sends a signal to B and C.

For the purposes of the exercise, when C encounters the smallest item in the list, the comparison returns "False". Also, in the first loop, i advances with a step of 2.

Complete the execution of this algorithm on the list $L = [6, 3, 1, 2, 5, 4]$ of size $n = 6$ on the following execution table, where 1 time unit allows to perform 1 operation.

(You will attach to the report a qualitative photo from the table below, filled in, for this question.)

Question 4 (/2)

Count the number of operations performed by each thread and the total number of operations (A + B + C operations) and compare it to the total number of operations needed for the sequential algorithm. What do you deduce? Which lines thread A could have executed to save operations?

Question 5 (/1)

Compare this time the execution time of sorting L with the time it would have taken to sort it completely sequentially. Give the report $\frac{\text{Time Algo Sequential}}{\text{Time Algo Parallel}}$, called speed-up.

Question 6 (/1)

What could we modify, in the parallel algorithm, to reduce execution time even more?

Question 7 (/2,5)

Give the time that the algorithm would have taken if we had used 3 secondary threads, then give the speed-up (start by counting the number of loops on i and on j). Same question for 4 secondary threads.

Do you think the result would have been the same if the list had been size 8? What do you deduce?

Question 8 (/0,5)

What would have happened if we had only used one secondary thread?

Question 9 (/1)

Give the expression of the time complexity of the algorithm for a list of size n and for m threads, with n very large and m then. Count above all the number of loops on i and on j .

Question 10 (/1)

The complexity of this parallel algorithm is in $\mathcal{O}\left(n + \frac{n^2}{m}\right)$. What is the value of m that optimizes complexity? Do you find this achievable?

1.2 Merge sort (/12)

The merge sort is a classic example of a divide and conquer algorithm. The principle is to divide the list to be sorted into 2 sub-lists, sort the sub-lists, then merge them by putting the elements in the correct order.

We define the following 3 algorithms :

Algorithm 2: SubLists

Data: L liste, n taille L , $a < n$ taille de $L1$

```
1  $L1, L2 \leftarrow []$ 
  for  $i = 0$  to  $a - 1$  do
     $L1[i] \leftarrow L[i]$ 
  for  $i = a$  to  $n - 1$  do
5    $L2[i - a] \leftarrow L[i]$ 
6 return  $L1, L2$ 
```

Algorithm 3: Merging

Data: $L1$ liste, $n1$ taille de $L1$, $L2$ liste, $n2$ taille de $L2$

```
1  $L \leftarrow []$ 
   $i, j \leftarrow 0$ 
   $L1[n1], L2[n2] \leftarrow \infty$ 
  while  $i \leq n1 - 1 \parallel j \leq n2 - 1$  do
5   if  $L1[i] < L2[j]$  then
     $L[i + j] \leftarrow L1[i]$ 
     $i \leftarrow i + 1$ 
  else
     $L[i + j] \leftarrow L2[j]$ 
10   $j \leftarrow j + 1$ 
11 return  $L$ 
```

Algorithm 4: Sort

Data: L liste, n taille de L

```
1 if  $n == 1$  then
  return  $L$ 
   $n1 \leftarrow \lfloor \frac{n}{2} \rfloor$ 
   $n2 \leftarrow \lceil \frac{n}{2} \rceil$ 
5  $L1, L2 \leftarrow \text{SousListes}(L, n, n1)$ 
   $L1\_Trie \leftarrow \text{Tri}(L1, n1)$ 
   $L2\_Trie \leftarrow \text{Tri}(L2, n2)$ 
   $L\_Trie \leftarrow \text{Fusion}(L1\_Trie, n1, L2\_Trie, n2)$ 
9 return  $L\_Trie$ 
```

Question 1 (/0,5)

Give the complexity in "big $\mathcal{O}$ " of the Algorithm 2.

Question 2 (/0,5)

Give the complexity in "big $\mathcal{O}$ " of the Algorithm 3 according to $n1$ and $n2$.

Question 3 (/1,5)

Give the complexity of lines 5, 6, 7 and 8 of the Algorithm 4. Deduce the expression from the number of operations of this algorithm, which is noted $T(n)$, according to $T(\frac{n}{2})$. (Warning : $T(n)$ is not the same thing as the notation $\mathcal{O}(n)$)

Question 4 (/1)

Using the *Master Theorem*, of which you can find the statement here², calculate the complexity of the Algorithm 4, by specifying the base of the logarithm.

Question 5 (/1,5)

Compare the complexity of merge sort with those of sequential and parallel selections. From how many secondary threads m is parallel selection sorting faster than merge sorting? What does it give for a list of size 100?

Question 6 (/1)

We are going to parallelize a merge sort. Firstly, with each passage in the function *Sort*, we will call 2 secondary keys to perform in parallel lines 6 and 7 of the Algorithm 4, then we will perform the fusion sequentially. Give the new complexity (calculate $T(n)$ then apply the Master Theorem).

Question 7 (/2)

How many threads would be launched in the first pass in *Sort*? In the 2nd and 3rd passes? In the n th? How many levels are there for *Sort* function calls? Deduce the number of secondary threads launched in total. Find several tips to use only n , then $\frac{n}{2}$ threads. (Reminder : $\sum_{i=1}^n a^i = \frac{a(a^n-1)}{a-1}$)

1.2.1 Question 8 (/1)

This time, at line 5 of *Sort*, we will split the list into m sequential sublists, then run m threads in parallel, and merge the sublists into sequential. Express then calculate $T(n)$. Also calculate the number of threads needed throughout the algorithm. Is it faster to split L into more than 2 sublists?

Question 9 (/2,5)

Our algorithm is still not convincing : it requires $m = n$ threads to work. By setting the number of threads m , explain or write in pseudo-code a parallel merge sort algorithm that would only use these secondary threads (modify only the "sort" part). Detail its complexity.

Question 10 (/0,5)

Suppose we have access to a large number of threads, which parts of *Sort* should now be changed to improve complexity?

2. https://fr.wikipedia.org/wiki/Master_theorem